

# One program, many models: AlgebraicDynamics and StockFlow

The same program on a composed resource sharer and a stock-and-flow diagram  
(Julia 1.11 environment)

AlgebraicDynamics.jl and StockFlow.jl currently resolve against an older Catlab than AlgebraicPetri.jl, so this companion to the main vignette has its own environment. It builds SIR as two composed resource sharers and as a stock-and-flow diagram, and applies the same program as the main document to both.

```
using CategoricalInterventions
using AlgebraicDynamics
using Catlab.Programs
using Catlab.WiringDiagrams: oapply
using ComponentArrays
using DiffEqCallbacks
using LabelledArrays
using OrdinaryDiffEq
using Plots
using StockFlow
using StockFlow.Syntax

default(; linewidth=2, grid=false)

infection_dynamics(u, p, t) = (rate = p.inf * u[1] * u[2]; [-rate, rate])
recovery_dynamics(u, p, t) = (rate = p.rec * u[1]; [-rate, rate])

infection = ContinuousResourceSharer{Float64}(2, infection_dynamics)
recovery = ContinuousResourceSharer{Float64}(2, recovery_dynamics)

sir_pattern = @relation (S, I, R) begin
    infection(S, I)
    recovery(I, R)
```

```

end
sir_system = oapply(sir_pattern, [infection, recovery])

```

The composed system keeps a positional state vector, so the model is told the state names in order. Parameters are labelled with ComponentArrays.

```

model = Model(sir_system; states=[:S, :I, :R], parameters=[:inf, :rec],
              invariants=[Conserved([:S, :I, :R])])

program = @interventions model begin
    lockdown    = scale(:inf, 0.5; during=5.0..25.0)
    vaccination = transfer(:S => :R, 50.0; at=15.0)
end

u0 = [990.0, 10.0, 0.0]
p0 = ComponentArray(inf=0.05 * 10.0 / sum(u0), rec=0.25)
tspan = (0.0, 40.0)
grid = 0.0:0.5:40.0

sol = simulate(model, program; u0, p0, tspan, alg=Tsit5(), saveat=grid)
sol[3, end], sum(sol.u[end])

```

```
(355.8150819160382, 1000.0)
```

## StockFlow

A stock-and-flow diagram becomes a model whose parameters are its parameters and whose states are its stocks. Labelled arrays supply the names to the solver.

```

sf = @stock_and_flow begin
    :stocks
    S
    I
    R

    :parameters
    inf
    rec

```

```

    :dynamic_variables
    v_infection = inf * S * I
    v_recovery = rec * I

    :flows
    S => f_infection(v_infection) => I
    I => f_recovery(v_recovery) => R

    :sums
    N = [S, I, R]
end
stockflow = Model(sf; invariants=[Conserved([:S, :I, :R])])

u0l = LVector(S=990.0, I=10.0, R=0.0)
p0l = LVector(inf=p0.inf, rec=p0.rec)
sf_sol = simulate(stockflow, program; u0=u0l, p0=p0l, tspan, alg=Tsit5(), saveat=grid)
sf_sol[3, end], maximum(abs, Array(sol) .- Array(sf_sol))

```

(355.8150819102329, 8.430106390733272e-9)

## Cross-check against the direct equations

```

sir_ode(u, p, t) = [-p.inf * u[1] * u[2], p.inf * u[1] * u[2] - p.rec * u[2], p.rec * u[2]]
direct = Model(sir_ode; parameters=[:inf, :rec], states=Dict{:S => 1, :I => 2, :R => 3})
direct_sol = simulate(direct, program; u0, p0, tspan, alg=Tsit5(), saveat=grid)
maximum(abs, Array(sol) .- Array(direct_sol))

```

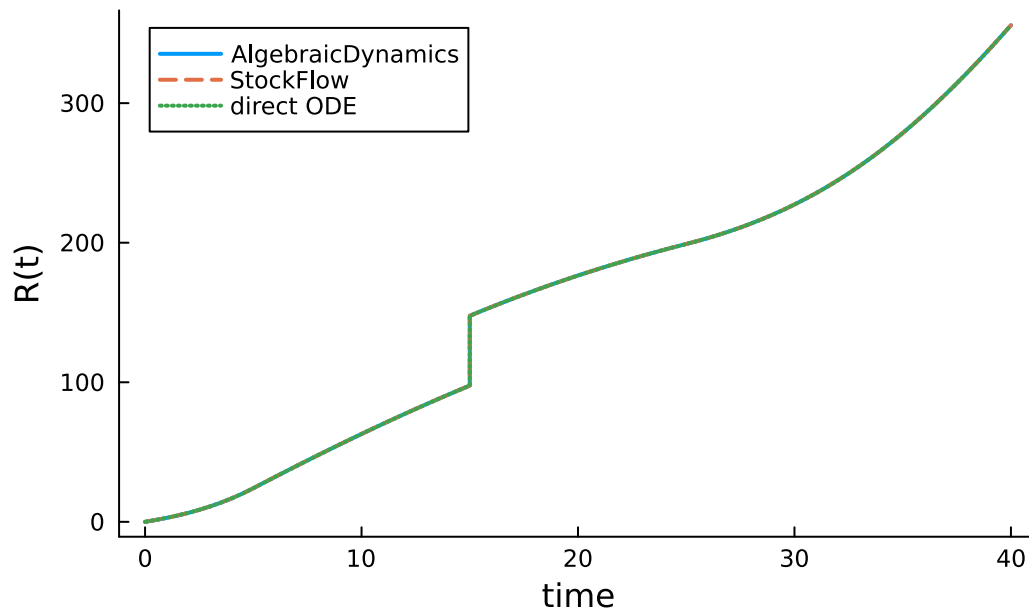
0.0

```

plot(sol.t, sol[3, :]; label="AlgebraicDynamics", xlabel="time", ylabel="R(t)",
     title="Same program: resource sharer, stock-and-flow, direct ODE")
plot!(sf_sol.t, sf_sol[3, :]; label="StockFlow", linestyle=:dash)
plot!(direct_sol.t, direct_sol[3, :]; label="direct ODE", linestyle=:dot)

```

time program: resource sharer, stock-and-flow, direct



### What the category theory bought

The same statement as in the main document: the program's meaning is fixed by its algebra and schedule before a model is chosen (`apply_pullback`, `lower_eq_apply`), so a composed resource sharer and a hand-written vector field, and a stock-and-flow diagram, receive identical callbacks.